

5장. 오차역전파법(3) 실습문제

신경망

학번: 202404252 이름: 원종우

1. 5.7.2 절(슬라이드 13)에서 TwoLayerNet 클래스에서 굵은 글씨로 표시된 부분이 세 군데 있다. 코드를 모두 옮겨 적고 각 줄마다 주석으로 각 줄의 설명(의미, 기능 등)을 추가하시오.

- (1) `__init__` 함수 부분: 신경망 구조 및 파라미터 초기화
- (2) `predict` 함수 부분: 순전파(Forward Propagation)
- (3) `gradient` 함수 부분: 역전파(Backward Propagation)

```
import numpy as np
from layers import *
from functions import numerical_gradient
from collections import OrderedDict

class TwoLayerNet:
    def __init__(self, input_size, hidden_size, output_size, weight_init_std = 0.01):
        # 신경망의 가중치와 편향을 저장할 딕셔너리 생성
        self.params = {}

        # 1층(입력층 → 은닉층)의 가중치 w1 초기화
        self.params['W1'] = weight_init_std * np.random.randn(input_size, hidden_size)
        # 1층의 편향 b1을 0으로 초기화
        self.params['b1'] = np.zeros(hidden_size)

        # 2층(은닉층 → 출력층)의 가중치 w2 초기화
        self.params['W2'] = weight_init_std * np.random.randn(hidden_size, output_size)
        # 2층의 편향 b2를 0으로 초기화
        self.params['b2'] = np.zeros(output_size)

        # 순서가 보장된 계층 저장용 OrderedDict 생성
        self.layers = OrderedDict()
        # 첫 번째 Affine(전결합) 계층 생성
        self.layers['Affine1'] = Affine(self.params['W1'], self.params['b1'])
        # 활성화 함수 계층 (ReLU)
        self.layers['Relu1'] = Relu()
        # 두 번째 Affine 계층 (은닉층 → 출력층)
        self.layers['Affine2'] = Affine(self.params['W2'], self.params['b2'])

        # 마지막 층: 소프트맥스 + 손실 함수 (교차 엔트로피)
        self.lastLayer = SoftmaxWithLoss()
```

```

def predict(self, x):
    # 입력 데이터를 받아 각 계층을 순서대로 통과시키며 출력 계산
    for layer in self.layers.values():
        x = layer.forward(x) # forward() 함수를 통해 Affine → ReLU → Affine 순으로 진행
    return x # 최종 출력(로짓 값, 즉 Softmax 이전 값)을 반환

def loss(self, x, t):
    y = self.predict(x)
    return self.lastLayer.forward(y, t)

def accuracy(self, x, t):
    y = self.predict(x)
    y = np.argmax(y, axis=1)
    if t.ndim != 1 : t = np.argmax(t, axis=1)

    accuracy = np.sum(y == t) / float(x.shape[0])
    return accuracy

def numerical_gradient(self, x, t):
    loss_W = lambda W: self.loss(x, t)

    grads = {}
    grads['W1'] = numerical_gradient(loss_W, self.params['W1'])
    grads['b1'] = numerical_gradient(loss_W, self.params['b1'])
    grads['W2'] = numerical_gradient(loss_W, self.params['W2'])
    grads['b2'] = numerical_gradient(loss_W, self.params['b2'])

    return grads

def gradient(self, x, t):
    # 순전파를 통해 손실 계산 (SoftmaxWithLoss 계층의 forward 호출)
    self.loss(x, t)

    # 역전파의 시작점: 손실에 대한 미분값은 항상 1
    dout = 1
    # SoftmaxWithLoss 계층의 역전파 수행 (예측과 정답 간의 오차 전파)
    dout = self.lastLayer.backward(dout)

    # 순전파 때의 층 순서를 뒤집어 역전파 순서대로 처리
    layers = list(self.layers.values())
    layers.reverse()

    # 각 층에서 backward() 실행하여 미분값을 역전파

```

```

for layer in layers:
    dout = layer.backward(dout)

# 각 층의 기울기(가중치 w, 편향 b)를 grads 딕셔너리에 저장
grads = {}
grads['W1'], grads['b1'] = self.layers['Affine1'].dW,
self.layers['Affine1'].db
grads['W2'], grads['b2'] = self.layers['Affine2'].dW,
self.layers['Affine2'].db

# 계산된 모든 기울기 반환
return grads

```

2. 5.7.3 절의 결론(알려 주고자 하는 내용)을 간단히 설명하시오.

오차역전파법은 빠르고 효율적으로 기울기를 계산할 수 있지만, 구현이 복잡해 오류가 생기기 쉽다. 따라서 수치 미분은 학습에 사용하기보다는, 오차역전파법이 올바르게 구현되었는지 검증(기울기 확인, gradient check)에 사용하는 것이 그 목적이다.

3. 5.7.4 절 코드에 대해 다음 물음에 답하시오.

(1) 이 코드의 동작 순서(절차, 흐름)를 글(비교적 편한 언어)로 설명하시오.

먼저 데이터를 불러와 정규화하고, 입력층 784개, 은닉층 50개, 출력층 10개로 구성된 신경망을 초기화한다. 그다음 정해진 학습 횟수만큼 반복하면서, 매번 학습 데이터 중 일부를 무작위로 뽑아 미니배치를 만든다. 이 데이터를 이용해 역전파로 가중치의 기울기를 계산하고, 학습률에 따라 가중치와 편향을 갱신한다. 손실값을 기록하면서 일정 주기마다 전체 학습 데이터와 테스트 데이터의 정확도를 계산해 출력한다. 이 과정을 통해 신경망이 점점 더 정확하게 숫자를 인식하도록 학습된다.

(2) loss와 accuracy를 누적하여 저장하였다. 저장한 이유는?

학습이 얼마나 잘 진행되고 있는 지 확인하기 위해서이다.

(3) 이 코드를 타이핑하고 실행하시오. 코드와 출력 결과를 캡처하여 붙여넣기 하시오.

```

python3 ques_3.py
train acc: 0.08686666666666666
test acc: 0.0872
train acc: 0.90215
test acc: 0.9063
train acc: 0.92355
test acc: 0.9258
train acc: 0.9374833333333333
test acc: 0.9368
train acc: 0.9456666666666667
test acc: 0.9458
train acc: 0.95125
test acc: 0.9507
train acc: 0.9576333333333333
test acc: 0.9541
train acc: 0.9617833333333333
test acc: 0.958
train acc: 0.96485
test acc: 0.96
train acc: 0.9674833333333334
test acc: 0.9631
train acc: 0.97015
test acc: 0.9643
train acc: 0.9713166666666667
test acc: 0.9666
train acc: 0.9737833333333333
test acc: 0.966
train acc: 0.9746
test acc: 0.9694
train acc: 0.97655
test acc: 0.9705
train acc: 0.97845
test acc: 0.9704
train acc: 0.9786166666666667
test acc: 0.9701

```